```
root@ant-Aspire-4736:/home/ant/pmd# ls
Makefile    Module.symvers  pmd.c  pmm.h   pmm.mod.c  pmm.o
modules.order  pmd         pmm.c  pmm.ko  pmm.mod.o
root@ant-Aspire-4736:/home/ant/pmd# insmod pmm.ko
root@ant-Aspire-4736:/home/ant/pmd# ls
Makefile    Module.symvers  pmd.c  pmm.h   pmm.mod.c  pmm.o
modules.order  pmd         pmm.c  pmm.ko  pmm.mod.o
root@ant-Aspire-4736:/home/ant/pmd# ./pmd
root@ant-Aspire-4736:/home/ant/pmd# Memhook-malloc: sz=100, caller=134516226,
result=153344904, file=pmd.c, func=main, line=207
root@ant-Aspire-4736:/home/ant/pmd# Memhook-realloc: sz=200, caller=134516286, result=153344904, file=pmd.c, func=main, line=209
Memhook-free: ptr=153344904, caller=134516338, file=pmd.c, func=main, line=211
Memhook-malloc: sz=100, caller=134516226, result=153344904, file=pmd.c, func=main, line=207
Memhook-realloc: sz=200, caller=134516286, result=153344904, file=pmd.c, func=main, line=209
Memhook-free: ptr=153344904, caller=134516338, file=pmd.c, func=main, line=211
Memhook-malloc: sz=100, caller=134516226, result=153344904, file=pmd.c, func=main, line=207
Memhook-realloc: sz=200, caller=134516286, result=153344904, file=pmd.c, func=main, line=209
Memhook-free: ptr=153344904, caller=134516338, file=pmd.c, func=main, line=211
Memhook-malloc: sz=100, caller=134516226, result=153344904, file=pmd.c, func=main, line=207
Memhook-realloc: sz=200, caller=134516286, result=153344904, file=pmd.c, func=main, line=209
Memhook-free: ptr=153344904, caller=134516338, file=pmd.c, func=main, line=211
Memhook-malloc: sz=100, caller=134516226, result=153344904, file=pmd.c, func=main, line=207
Memhook-realloc: sz=200, caller=134516286, result=153344904, file=pmd.c, func=main, line=209
Memhook-free: ptr=153344904, caller=134516338, file=pmd.c, func=main, line=211
Memhook-malloc: sz=100, caller=134516226, result=153344904, file=pmd.c, func=main, line=207
Memhook-realloc: sz=200, caller=134516286, result=153344904, file=pmd.c, func=main, line=209
Memhook-free: ptr=153344904, caller=134516338, file=pmd.c, func=main, line=211
```

Fig. 1: Screenshot of the program output

EFY Note

The source codes of this project are included in this month's EFY DVD and are also available for free download at source.efymag.com.

Start pmthread, a kernel thread. The thread runs over the kernel task structure to find out whether current monitoring user space daemons are in active or killed state.

Note. The kernel thread can be enhanced to process memory footprint at any point by accessing vm and rss sizes for understanding its overall increase in usage against the time quantum.

For further development

Check for other data members of kernel task_struct structure of user space processes for getting the complete picture of vm and rss sizes, shared library size, CS and DS segment sizes and much more. Using simple math, we can find out the overall memory consumption of processes that can be used for memory-leak alerting.

Hook functions can be separated as shared library so that these become a ready-made solution for developers seeking a programmable memory-leak logger infrastructure.

Testing procedure

Steps to test involve compilation and verification as a root user from the terminal. Follow the sequence as given below.

1. For compiling our user daemon, issue the following command:

```
# gcc pmd.c -o pmd -g -I .
```

2. For compiling the kernel module (kernel makefile is included):

```
# make
```

3. For installing the module, issue the following command:

```
# insmod pmm.ko
```

4. For starting the daemon, issue the following command:

```
# ./pmd
```

A typical output of this command run in Ubuntu system is shown in the screenshot above (Fig. 1).

5. For viewing memory request logs, issue the following command:

```
# cat /proc/pmm/pid
```

Sample output

Some sample outputs are given below. Issue following commands on the terminal:

```
# cat /proc/pmm/pid
```

The output will be something like this:

```
[1] PID: 21113, Comp: pmd, PPID: 1,
State: 1, Log count: 2
```

This means, the pmm proc is registered by the kernel module. It displays the format of pid file memory logs. (Here, PID of 'pmd' process is 21113.)

Issue following commands on the terminal:

```
# ./pmd
```

The output will be something like this:

```
(MALLOC @06:57:11) Sz: 100B Pointer:
0x10012008 Caller: 0x10000e60 main()
207 pmd.c
(REALLOC @06:57:21) Sz: 200B
Prev Pointer: 0x10012008 Pointer:
0x10012008 Caller: 0x10000e84 main()
209 pmd.c
```

The meanings of these outputs are given below.

type of log. {< malloc/ realloc > @ < system_time > }

Sz. < Size of the block allocated in Bytes >

Pointer. < Memory pointer within the running process data area >

Caller. < Process code area from where it got allocated >

Location. function-name, line-number and file-name.

The above details will give user the system status and health conditions.

To stop the process, issue following commands:

```
#ps -ef | grep pmd
root 20604 2 0 06:27 ? 00:00:00
[pmthread]
root 21113 1 0 06:30 ? 00:00:00 ./pmd
```

Issue the following commands on the terminal:

```
# cat /proc/pmm/cfg
```

The output will be something like this:

```
Configs:
["INTERVAL" (Scan Interval) :=10000]
["SCAN"  (Scan on:1/off:0)  := 1]
["START_PID" (Start a monitor):=21113]
["STOP_PID" (Stop a monitor) := 0]
```

This output displays configuration objects including interval, scanning and start-and-stop status.

For exanokem INTERVAL configuration is currently set to 10000ms and this can be changed using the following command:

```
#Printf "INTERVAL 5000" >> /proc/pmm/cfg
```

To start monitoring on a PID, use the following command:

```
#Printf "START_PID <pid_num>>> /proc/
pmm/cfg
```



L. Karthikeyan is B.E. (computer science) from AC Tech, Karaikudi. He is working as a technical lead, HCL Technologies, Chennai. He likes to read and write articles related to computer science and finance